

APBD 2026 | Ćwiczenia 7

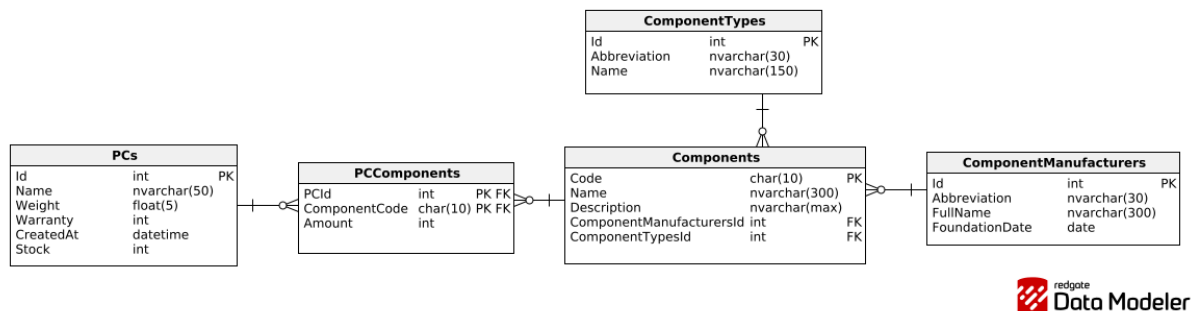
Poniżej znajduje się treść ćwiczenia.

REST API do zarządzania komputerami i komponentami z wykorzystaniem EF Core Code First

Celem zadania jest utworzenie aplikacji **REST Web API** służącej do zarządzania komputerami oraz ich komponentami. Projekt zostanie zrealizowany z wykorzystaniem platformy ASP.NET Core oraz **Entity Framework Core** w podejściu **Code First**. Na podstawie zdefiniowanych klas encji zostanie automatycznie utworzona struktura bazy danych wraz z relacjami między tabelami.

Code First

Na podstawie przedstawionego diagramu bazy danych należy przygotować model aplikacji w podejściu **Entity Framework Core Code First**. W tym celu należy utworzyć odpowiednie klasy encji odwzorowujące tabele z diagramu oraz skonfigurować relacje pomiędzy nimi, takie jak relacje jeden-do-wielu oraz wiele-do-wielu.



Dodatkowo należy zaimplementować wymagane ograniczenia i konfiguracje, między innymi:

- klucze główne,
- klucze obce,
- wymagane pola,
- maksymalne długości danych tekstowych,
- typy danych,
- klucze złożone,
- oraz odpowiednie właściwości nawigacyjne.

Po przygotowaniu modeli należy utworzyć kontekst bazy danych (**AppDbContext**) oraz wygenerować strukturę bazy danych przy użyciu **migracji** Entity Framework Core.

Dodatkowo baza danych powinna zostać wypełniona przykładowymi danymi startowymi (seed data). Należy przygotować **minimum 3 rekordy** dla każdej tabeli, z zachowaniem poprawnych

relacji pomiędzy encjami. Seedowanie danych powinno zostać zrealizowane przy użyciu mechanizmu `HasData()` lub innego rozwiązania dostępnego w Entity Framework Core.

Zakres endpointów

- GET `api/pcs`
- GET `api/pcs/{id}/components`
- POST `api/pcs`
- PUT `api/pcs/{id}`
- DELETE `api/pcs/{id}`

Logika endpointów

GET `api/pcs`

Endpoint zwraca listę wszystkich komputerów wraz z podstawowymi informacjami, takimi jak nazwa, waga, okres gwarancji, data utworzenia oraz stan magazynowy.

Przykładowa odpowiedź `GetAllResponse.json`:

```
[
  {
    "id": 1,
    "name": "Gaming Beast X",
    "weight": 12.5,
    "warranty": 36,
    "createdAt": "2026-05-08T09:00:00",
    "stock": 5
  },
  {
    "id": 2,
    "name": "Office Mini Pro",
    "weight": 4.2,
    "warranty": 24,
    "createdAt": "2026-04-15T13:30:00",
    "stock": 12
  }
]
```

GET `api/pcs/{id}/components`

Endpoint zwraca listę komponentów przypisanych do wybranego komputera o podanym identyfikatorze. Wynik powinien zawierać informacje o komponentach oraz ich ilości w danym zestawie komputerowym. Jeżeli komputer nie istnieje, zwróć `404 Not Found`.

Przykładowa odpowiedź znajduje się w pliku `GetByIdResponse.json`.

POST api/pcs

Endpoint powinien umożliwiać dodanie nowego komputera. Dane komputera powinny zostać przekazane w treści żądania, a po poprawnym zapisaniu rekord powinien zostać dodany do bazy.

Przykładowa zawartość body żądania `PostRequest.json`:

```
{
  "name": "Gaming Beast X",
  "weight": 12.5,
  "warranty": 36,
  "createdAt": "2026-05-08T09:00:00",
  "stock": 5
}
```

Przykładowa odpowiedź `PostResponse.json`:

```
{
  "id": 1,
  "name": "Gaming Beast X",
  "weight": 12.5,
  "warranty": 36,
  "createdAt": "2026-05-08T09:00:00",
  "stock": 5
}
```

PUT api/pcs/{id}

Endpoint umożliwia edycję danych istniejącego komputera na podstawie podanego identyfikatora. Zmodyfikowane informacje przekazane w żądaniu powinny zostać zapisane w bazie danych.

Przykładowa zawartość body żądania `PutRequest.json`:

```
{
  "name": "Gaming Beast X",
  "weight": 12.5,
  "warranty": 36,
  "createdAt": "2026-05-08T09:00:00",
  "stock": 5
}
```

DELETE api/pcs/{id}

Endpoint powinien umożliwiać usunięcie komputera o wskazanym identyfikatorze z bazy danych wraz z powiązaniami, jeśli zostały odpowiednio skonfigurowane relacje w Entity Framework Core. Jeśli komputer nie istnieje, zwróć **404 Not Found**. Po poprawnym usunięciu zwróć **204 No Content**.

Pamiętaj o:

Podczas realizacji projektu należy zwrócić szczególną uwagę na dobre praktyki programistyczne oraz poprawną organizację aplikacji.

- Zastosuj odpowiednią architekturę z podziałem na warstwy, pamiętając aby rozdzielić aplikację na warstwę:
 - logiki biznesowej,
 - oraz warstwę API/prezentacji.
- Nie udostępniaj bezpośrednio encji bazy danych w endpointach API. Przygotuj osobne klasy **DTO (Data Transfer Object)** dla:
 - danych przychodzących do aplikacji,
 - danych zwracanych przez API,
 - oraz operacji tworzenia i aktualizacji danych.
- Skonfiguruj oraz wykorzystaj **migracje** Entity Framework Core do tworzenia i aktualizacji struktury bazy danych. Wszystkie wygenerowane migracje powinny zostać dodane do repozytorium projektu.
- Endpointy API powinny zwracać odpowiednie statusy HTTP, zgodne z rezultatem wykonywanej operacji, między innymi:
 - **200 OK** - poprawne pobranie lub modyfikacja danych,
 - **201 Created** - poprawne utworzenie zasobu,
 - **204 No Content** - poprawne usunięcie danych,
 - **400 Bad Request** - niepoprawne dane wejściowe,
 - **404 Not Found** - brak wskazanego zasobu,
 - **500 Internal Server Error** - błąd serwera/aplikacji.
- Zadbaj o poprawną konfigurację relacji oraz integralność danych w bazie danych.
- Kod projektu powinien być czytelny, odpowiednio podzielony na katalogi oraz zgodny z konwencjami nazewnictwa języka C# i platformy ASP.NET Core.
- W projekcie należy stosować operacje asynchroniczne (**async/await**) podczas komunikacji z bazą danych.